Sanket Ajay Chopade

DevOps Engineer

Git Branching Strategy

Complete Interview Q&A Reference Guide

AWS · Kubernetes · Terraform · Jenkins · GitHub Actions · Argo CD · EKS

20 Sections · 100+ Questions with Structured Answers

Section 1 · Git Fundamentals

Q. What is Git?

- Git is a distributed version control system that tracks changes in source code over time.
- Multiple developers collaborate on the same codebase without overwriting each other's work.
- Git stores snapshots of the entire project, not just file differences.
- Works offline; changes sync to a remote repository when connectivity is available.

Q. What is a branch in Git?

- A branch is a lightweight, movable pointer to a specific commit in the repository history.
- It allows developers to diverge from the main line of development and work in isolation.
- Branches are cheap to create and fast to switch because they are just pointers to commits.
- Common branches: main, develop, feature/*, release/*, hotfix/*.

Q. Why do we use branching?

- Isolates new features, bug fixes, and experiments from stable production code.
- Enables parallel development by multiple team members without interference.
- Supports structured code review workflows via pull requests before merging.
- Provides a clear audit trail of what was changed, why, and when.

Q. What is the default branch in Git?

- Historically 'master'; modern Git and GitHub/GitLab default to 'main'.
- Represents the primary line of development.
- In a GitOps setup (Argo CD), this is the source of truth for production deployments.

Q. Difference between Git and GitHub

- Git is the version control tool installed locally on a developer's machine.
- GitHub is a cloud-based hosting platform for Git repositories.
- GitHub adds collaboration features: pull requests, issues, Actions (CI/CD), branch protection.
- Alternatives: GitLab, Bitbucket, AWS CodeCommit.

Q. What is a commit?

- A commit is a saved snapshot of the repository at a specific point in time.
- Each commit has a unique SHA-1 hash, an author, a timestamp, and a commit message.
- Best practice: small, atomic commits with clear messages (e.g., 'feat: add payment endpoint').
- Commits are immutable; changing history requires rewriting with rebase or reset.

Q. What is a merge?

- Merging integrates changes from one branch into another.
- A merge commit has two parents, preserving the full history of both branches.
- Types: fast-forward merge (linear history), 3-way merge (creates a merge commit), squash merge.

Q. What is a rebase?

- Rebase moves or re-applies commits from one branch onto another, rewriting history.
- Produces a linear commit history, making the log cleaner and easier to read.

- Golden rule: never rebase commits that have already been pushed to a shared/public branch.
- Useful for cleaning up local feature branch commits before opening a PR.

Q. What is a pull request?

- A PR is a formal request to merge changes from one branch into another.
- It triggers code review, CI/CD pipeline execution, and team discussion.
- PRs are the primary gate for enforcing code quality and branch protection rules.
- In GitHub Actions, a PR event can trigger tests, security scans (Trivy, SonarQube), and linting.

Q. What is a fork?

- A fork is a personal copy of someone else's repository under your own GitHub account.
- Common in open-source workflows; changes are proposed back via pull requests.
- In enterprise teams, forking is less common; feature branches within the same repo are preferred.

Section 2 · Branching Strategy Basics

Q. What is a Git branching strategy?

- A defined convention for how branches are created, named, merged, and deleted.
- It standardises the flow of code from development through testing to production.
- Popular strategies: GitFlow, GitHub Flow, GitLab Flow, Trunk-Based Development.

Q. Why is a branching strategy important?

- Prevents code conflicts and unstable deployments to production.
- Enables parallel feature development without blocking releases.
- Aligns CI/CD pipelines with deployment gates (dev, QA, UAT, prod).
- Makes rollbacks predictable and auditable.

Q. What problems can occur without a branching strategy?

- Developers directly committing to main, breaking production builds.
- Merge conflicts accumulating because branches diverge for too long.
- No clear gate for code review, testing, or security scanning.
- Inability to isolate a hotfix from unfinished features.

Q. What branching strategy have you used?

- At Hisan Labs, we followed a Feature Branch Workflow aligned with GitFlow principles.
- Feature branches were created from develop, tested in isolation, and merged via pull requests.
- For the GitOps project, I used a two-repo model: app repo and Kubernetes manifest repo, with Argo CD watching the main branch of the manifest repo.

Q. Which branching strategy for a startup?

- GitHub Flow or Trunk-Based Development — both prioritise speed and simplicity.
- Small teams benefit from a single long-lived branch (main) with short-lived feature branches.
- Feature flags replace long-lived release branches, enabling continuous deployment.

Q. Which branching strategy for a large enterprise?

- GitFlow or GitLab Flow — structured, supports multiple parallel releases and hotfixes.
- Environment branches (dev, staging, production) map directly to deployment targets.
- Release trains and gated approvals are easier to implement with explicit release branches.

Q. How does branching strategy impact CI/CD pipelines?

- Branch naming conventions determine which pipeline stage is triggered.
- Example: push to 'feature/*' triggers unit tests; merge to 'main' triggers full build + deploy.
- In GitHub Actions, 'on: push: branches: [main]' or 'on: pull_request' wires branches to workflows.
- In Jenkins, Multibranch Pipelines auto-discover branches and run the appropriate Jenkinsfile stage.

Section 3 · GitFlow

Q. What is GitFlow?

- GitFlow is a branching model introduced by Vincent Driessen in 2010.
- It defines strict branch roles: main, develop, feature/*, release/*, hotfix/*.
- Designed for projects with a scheduled release cycle and multiple parallel versions.

Q. What branches exist in GitFlow?

- main — production-ready code; every commit here represents a release.
- develop — integration branch for completed features; always deployable to staging.
- feature/* — branched from develop; merged back to develop when complete.
- release/* — branched from develop; final QA and bug fixes; merged to main AND develop.
- hotfix/* — branched from main; emergency fix; merged to main AND develop.

Q. Walk me through a new feature development using GitFlow.

- 1. Branch off develop: git checkout -b feature/payment-gateway develop
- 2. Develop and commit changes locally with descriptive commit messages.
- 3. Open a PR targeting develop; CI runs SonarQube, unit tests, Trivy scans.
- 4. After approval and all checks pass, squash-merge into develop.
- 5. Delete the feature branch after merging.

Q. How does a release move to production?

- 1. Branch from develop: git checkout -b release/1.3.0 develop
- 2. Run final QA; fix release-specific bugs; update version numbers.
- 3. Merge into main and tag: git tag -a v1.3.0 -m 'Release 1.3.0'
- 4. Merge release branch back into develop to carry forward any fixes.
- 5. CI/CD pipeline on main triggers deployment to production.

Q. How are emergency fixes handled?

- 1. Branch from main: git checkout -b hotfix/critical-auth-bug main
- 2. Apply the minimal fix needed; write a regression test.
- 3. Merge into main (tagged) AND into develop to keep branches in sync.
- 4. Trigger CI/CD pipeline on main for immediate production deployment.

Q. Advantages of GitFlow

- Clear separation of concerns between features, releases, and hotfixes.
- Supports multiple release versions in parallel.
- Strong audit trail for production deployments.
- Works well with scheduled release cycles (bi-weekly sprints, quarterly releases).

Q. Disadvantages of GitFlow

- Overhead of maintaining 5+ branch types is complex for small teams.
- Long-lived feature branches increase merge conflict risk.
- Slow feedback loop; code might not reach production for weeks.
- Not ideal for continuous deployment — better suited for less frequent shipping cycles.

Section 4 · Feature Branch Workflow

Q. What is Feature Branch Workflow?

- Every new feature or bug fix is developed in a dedicated branch isolated from main.
- Branches are short-lived, created from main/develop and deleted after merging.
- Code changes are reviewed and merged via pull requests.

Q. Why is it popular?

- Simple mental model: one feature = one branch.
- Encourages code review, improving quality and knowledge sharing.
- CI can run independently per branch, giving fast feedback to developers.
- Works seamlessly with GitHub, GitLab, Bitbucket PR workflows.

Q. How does a developer create a feature branch?

- git checkout main && git pull origin main
- git checkout -b feature/add-user-auth
- Develop, commit with meaningful messages, push to remote.
- Open a PR against main or develop; assign reviewers.

Q. When should a feature branch be deleted?

- Immediately after the PR is merged — keeps the repository clean.
- GitHub and GitLab both offer 'auto-delete branch after merge' settings.
- Stale unmerged branches should be cleaned up after ~30 days of inactivity.

Q. Challenges with long-lived feature branches

- Merge conflicts compound as main diverges over time — known as 'merge hell'.
- Integration issues discovered late, close to the release deadline.
- CI feedback is out-of-date because the branch doesn't reflect current main.
- Mitigation: merge/rebase from main frequently; use feature flags for incomplete work.

Section 5 · Trunk-Based Development (TBD)

Q. What is Trunk-Based Development?

- All developers commit to a single shared branch called trunk (main) frequently — at least once a day.
- Feature branches exist but are very short-lived (hours to a day, not weeks).
- Emphasises keeping the trunk always in a releasable state.

Q. How is TBD different from GitFlow?

- GitFlow: 5+ branch types, long-lived release branches, infrequent merges to main.
- TBD: one primary branch, very short-lived feature branches, continuous integration to trunk.
- TBD enables Continuous Deployment; GitFlow is better for versioned release cycles.

Q. Why is TBD popular in DevOps?

- Aligns perfectly with CI/CD — every commit to trunk can potentially be deployed.
- Reduces merge conflicts because integration happens frequently.
- Forces teams to keep code in a working state at all times.
- Used by Google, Facebook, and most organisations practising high-frequency deployment.

Q. How does CI/CD benefit from TBD?

- Every push to trunk triggers a full CI pipeline: build, test, scan, deploy to dev.
- No need to maintain pipeline definitions for multiple long-lived branches.
- Deployment frequency increases; mean time to recovery (MTTR) decreases.
- In GitHub Actions, a single 'on: push: branches: [main]' trigger covers the entire flow.

Q. What role do feature flags play?

- Feature flags decouple code deployment from feature release.
- Incomplete features can be merged to trunk but hidden behind a flag (enabled: false).
- When the feature is ready, the flag is toggled on — no code deployment needed.
- Tools: LaunchDarkly, AWS AppConfig, custom environment variables.

Q. Advantages of TBD

- Faster feedback loops; bugs caught earlier.
- No merge hell from long-lived branches.
- Simpler CI/CD pipeline design.
- Supports Continuous Deployment with high confidence.

Q. Disadvantages of TBD

- Requires mature CI/CD with fast, reliable test suites.
- Needs discipline: incomplete code must be gated behind feature flags.
- Not ideal for teams without strong automated testing coverage.
- Challenging in regulated environments requiring formal release approval gates.

Section 6 · GitHub Flow

Q. What is GitHub Flow?

- A simplified branching model: one main branch plus short-lived feature branches.
- Steps: branch from main, commit changes, open PR, review, merge, deploy.
- No develop, release, or hotfix branches — simplicity is the primary goal.

Q. How is GitHub Flow different from GitFlow?

- GitHub Flow: 2 branch types (main + feature); deploys after every merge to main.
- GitFlow: 5 branch types; deployments happen at scheduled release points.
- GitHub Flow suits teams doing continuous deployment; GitFlow suits versioned releases.

Q. Why is GitHub Flow suitable for continuous deployment?

- Every merge to main is immediately deployable — the pipeline runs and deploys automatically.
- Short-lived branches mean integration issues are found quickly.
- PRs provide a lightweight but effective gate for review and CI checks.
- Works well with GitHub Actions triggered on pull_request and push to main events.

Q. Typical GitHub Flow workflow

- 1. Create branch from main: git checkout -b feature/improve-search
- 2. Commit with descriptive messages.
- 3. Open PR: CI runs tests, lint, security scans.
- 4. Team reviews; required approvals enforced by branch protection.
- 5. Merge to main; deployment pipeline triggers automatically.
- 6. Delete the branch.

Section 7 · GitLab Flow

Q. What is GitLab Flow?

- GitLab Flow combines GitHub Flow simplicity with environment branches for structured deployments.
- Core branches: main (source of truth) plus environment branches: pre-production, production.
- Merges flow downstream: main, then pre-production, then production.

Q. What are environment branches?

- Branches that mirror a deployment environment: staging, pre-prod, production.
- Code is promoted by merging forward through the chain — never backwards.
- Example: main auto-deploys to staging; production requires a manual approval to merge.

Q. Why is GitLab Flow useful for DevOps teams?

- Bridges the gap between fast feature development (GitHub Flow) and environment control (GitFlow).
- Environment branches map directly to Kubernetes namespaces or AWS accounts.
- In an Argo CD setup, each environment branch can be a separate Argo CD Application.
- Provides clear promotion gates without the overhead of GitFlow release branches.

Section 8 · Branch Protection

Q. What is branch protection?

- A set of rules enforced by GitHub/GitLab that restrict direct changes to critical branches.
- Configured under Repository Settings > Branches > Branch Protection Rules.
- Can be applied to main, develop, release/*, or any pattern.

Q. Why protect the main branch?

- Prevents accidental force pushes that rewrite production history.
- Enforces pull request reviews before any code reaches production.
- Ensures the CI pipeline passes before merge, catching broken builds early.
- Maintains deployment integrity in a GitOps environment where Argo CD uses main as source of truth.

Q. What branch protection rules have you configured?

- Require pull request with at least 1 approved review before merging.
- Require status checks to pass: CI pipeline, SonarQube quality gate, Trivy scan.
- Dismiss stale approvals when new commits are pushed.
- Restrict who can push directly to main (only the CI service account).
- Require signed commits for audit compliance.

Q. How do you prevent direct pushes to production branches?

- Enable 'Require a pull request before merging' rule.
- Disable 'Allow force pushes' and 'Allow deletions'.
- Add CODEOWNERS file to auto-assign reviewers for critical paths.
- Use required status checks — the merge button is disabled until CI passes.

Section 9 · Pull Requests

Q. What is a Pull Request?

- A PR is a request to merge changes from a source branch into a target branch.
- It is the primary collaboration and code review mechanism in modern Git workflows.
- PRs provide a discussion thread, change diff view, CI results, and reviewer approvals.

Q. Why are Pull Requests important?

- Enforce code review as a mandatory gate before changes reach main.
- Trigger automated CI/CD pipelines: tests, scans, builds.
- Create a documented history of what changed, why, and who approved it.
- Enable asynchronous collaboration across time zones.

Q. What should be reviewed before approving a PR?

- Code correctness and logic: does it do what the description claims?
- Security: any hardcoded secrets, SQL injection risks, or unvalidated inputs?
- Test coverage: are new code paths covered by unit or integration tests?
- Infrastructure: Terraform plan output reviewed for unintended resource changes.
- Kubernetes manifests: resource limits set, health probes configured, correct namespace.

Q. What is the ideal PR size?

- Smaller is better — ideally under 400 lines of changed code.
- Large PRs are harder to review, more error-prone, and slower to merge.
- Break large features into incremental PRs; use feature flags to hide incomplete work.
- Rule of thumb: a PR should be fully reviewable in under 30 minutes.

Q. How do CI/CD pipelines integrate with PRs?

- GitHub Actions: 'on: pull_request: branches: [main, develop]' triggers the workflow.
- Pipeline runs: lint, unit tests, SonarQube analysis, Docker build, Trivy vulnerability scan.
- Required status checks block merge until all jobs pass.
- After merge, a separate 'on: push: branches: [main]' workflow triggers deployment.

Section 10 · Merge Strategies

Q. What is a merge commit?

- A merge commit has two parent commits — one from each branch being merged.
- Preserves the full commit history of both branches.
- Best for: teams that want a complete audit trail of when branches diverged and merged.
- Disadvantage: produces a noisy history with many 'Merge branch X into Y' commits.

Q. What is squash merge?

- Squash merge combines all commits from a feature branch into a single commit on the target branch.
- The feature branch's granular history is collapsed into one clean, descriptive commit.
- Best for: keeping main/develop clean and readable.
- Used at Hisan Labs to merge feature branches — one commit per feature on develop.

Q. What is rebase merge?

- Rebase applies commits from the feature branch on top of the target branch one by one.
- Produces a perfectly linear, clean history without merge commits.
- Best for: teams that prefer a linear git log (git log --oneline).
- Risk: rewrites commit SHAs — never rebase shared or public branches.

Q. Difference between merge and rebase

- Merge: non-destructive; preserves branch history; creates a merge commit.
- Rebase: rewrites history; produces linear commits; no merge commit created.
- Merge is safer for shared branches; rebase is better for cleaning up local commits.
- Best practice: rebase locally to clean commits before opening a PR, then squash-merge into develop.

Q. Risks of rebase

- Rewriting commits that others have already pulled breaks their local history.
- Force-pushing a rebased branch (git push --force-with-lease) is required but risky.
- Golden rule: only rebase your own local commits that have not yet been pushed to a shared branch.

Section 11 · Merge Conflicts

Q. What causes merge conflicts?

- Two branches modify the same line(s) in the same file differently.
- One branch deletes a file that another branch has modified.
- Binary files cannot be auto-merged by Git.
- Long-lived branches that diverge significantly from main are the most common cause.

Q. How do you resolve merge conflicts?

- 1. Identify conflicting files: git status shows 'both modified'.
- 2. Open the file — Git marks conflicts with <<<<<<<, =======, and >>>>>>>.
- 3. Decide which change to keep (yours, theirs, or a combination).
- 4. Remove conflict markers and stage the file: git add .
- 5. Complete the merge: git merge --continue or git commit.
- Tools used: VS Code built-in merge editor, IntelliJ IDEA, vimdiff, git mergetool.

Q. How do you reduce merge conflicts?

- Keep branches short-lived — merge to main within days, not weeks.
- Pull from main frequently: git pull --rebase origin main.
- Communicate within the team to avoid simultaneous edits to the same modules.
- Use modular code architecture to reduce overlapping file changes.
- Feature flags allow merging incomplete code early, keeping branches short.

Q. How would you handle conflicts in a large team?

- Enforce short-lived branches via team policy or stale branch automation alerts.
- Use CODEOWNERS to ensure the right people review changes to shared files.
- Trunk-Based Development with feature flags is the most effective long-term solution.
- Automated conflict detection tools in CI — alert early before PRs become unmergeable.

Section 12 · CI/CD Integration

Q. How do Git branches trigger Jenkins pipelines?

- Jenkins Multibranch Pipeline auto-scans the repository and creates a pipeline per branch.
- A Jenkinsfile in the repo root defines stages conditionally: 'when { branch main }' runs deploy.
- GitHub webhooks (configured at Hisan Labs) notify Jenkins on push events — triggering pipelines instantly.
- Branch filters restrict which branches trigger which stages (e.g., only release/* deploys to UAT).

Q. How do GitHub Actions workflows trigger?

- 'on: push: branches: [main]' — fires on every push to main.
- 'on: pull_request: branches: [main, develop]' — fires when a PR targets these branches.
- 'on: workflow_dispatch' — manual trigger from the GitHub UI.
- Multiple triggers can be combined in a single workflow YAML file.

Q. How does Argo CD work with Git branches?

- Argo CD Application resources specify a 'targetRevision' — this can be a branch, tag, or commit SHA.
- Argo CD polls the repository (or receives a webhook) and syncs when the target branch changes.
- Best practice: Argo CD watches 'main' for production; a 'staging' branch for the staging environment.
- Auto-sync is enabled for non-production; production requires manual sync approval.

Q. Which branch is the source of truth in GitOps?

- The branch that Argo CD's Application resource points to — typically 'main' for production.
- Any manual change to a Kubernetes resource will be detected as OutOfSync and reverted by Argo CD.
- This enforces immutable infrastructure: Git is always authoritative over the cluster state.

Q. How are environment-specific deployments handled?

- Separate Argo CD Applications per environment, each pointing to a different overlay (Kustomize) or values file (Helm).
- Promotion: update the image tag in the manifest repo and commit — Argo CD picks it up.
- In the GitOps pipeline project, GitHub Actions updated the EKS manifest after building the Docker image, then Argo CD synced the cluster.

Section 13 · GitOps Branching (Argo CD)

Q. How does GitOps use Git repositories?

- Git is the single source of truth for both application code and infrastructure state.
- Desired state is declared in Git; an operator (Argo CD) continuously reconciles the cluster to match.
- No manual kubectl apply — all changes go through Git commits and pull requests.

Q. Two-repo model: app repo vs config repo

- App Repo: application source code, Dockerfile, unit tests, CI pipeline (GitHub Actions).
- Config/Manifest Repo: Kubernetes manifests, Helm charts, Kustomize overlays — watched by Argo CD.
- CI in the app repo builds and pushes the Docker image, then updates the image tag in the manifest repo.
- Argo CD detects the config repo change and syncs the cluster automatically.

Q. How does Argo CD track branch changes?

- Argo CD Application 'targetRevision: main' tracks the HEAD of main.
- It polls every 3 minutes by default, or receives a webhook for instant sync.
- Any commit to the watched branch triggers a sync if auto-sync is enabled.
- Sync status: Synced (matches Git), OutOfSync (drift detected), Degraded (health issue).

Q. How are rollbacks performed in GitOps?

- Method 1 (preferred): revert the commit in Git — git revert HEAD — Argo CD syncs back.
- Method 2: in Argo CD UI, select History and Rollback to a previous Git revision.
- Method 3: update the image tag in the manifest repo to the last known good version.
- Unlike kubectl rollout undo, GitOps rollbacks are always traceable in Git history.

Q. What happens if someone manually changes a Kubernetes resource?

- Argo CD marks the Application as OutOfSync — live cluster diverges from Git.
- With auto-sync enabled, Argo CD reverts the manual change within minutes.
- This prevents configuration drift and enforces immutable infrastructure.
- Alerts can be configured to notify the team when drift is detected (Slack, PagerDuty).

Section 14 · Release Management

Q. How do you prepare a production release?

- 1. Cut a release branch from develop: git checkout -b release/2.1.0 develop
- 2. Freeze features; only bug fixes and documentation updates are permitted.
- 3. Run full regression tests (automated + manual UAT).
- 4. Merge into main and tag: git tag -a v2.1.0 -m 'Release 2.1.0'
- 5. CI/CD pipeline deploys the tagged version to production.

Q. How do you version releases?

- Semantic Versioning (SemVer): MAJOR.MINOR.PATCH — e.g., v2.1.3
- MAJOR: breaking API changes. MINOR: new backwards-compatible features. PATCH: bug fixes.
- Git tags mark releases: git tag -a v2.1.0 -m 'message' && git push origin v2.1.0
- Docker images are tagged with the Git tag: docker build -t app:v2.1.0 .

Q. How do you rollback a release?

- Git revert: creates a new commit that undoes the release changes — safe for shared history.
- In GitOps: revert the manifest repo commit; Argo CD syncs the cluster to the previous state.
- Kubernetes rollout undo: kubectl rollout undo deployment/app (last resort; not GitOps-compliant).
- Always rollback via Git so the action is auditable and traceable.

Q. How do you handle multiple simultaneous releases?

- Maintain separate release branches: release/2.1.0 and release/2.2.0 in parallel.
- Hotfixes to 2.1.0 production are cherry-picked to 2.2.0 if needed.
- Separate CI/CD pipeline branches handle each release track independently.

Section 15 · Hotfix Workflow

Q. What is a hotfix?

- An emergency fix applied directly to the production codebase to resolve a critical bug.
- It bypasses the normal feature development cycle to minimise production downtime.
- Must be merged back into both main AND develop to prevent regression in future releases.

Q. Walk me through a production outage caused by a bug.

- 1. Alert triggered in Grafana/Datadog — high error rate on the /payment endpoint.
- 2. Identify the faulty commit: git log --oneline main
- 3. Create hotfix branch: git checkout -b hotfix/payment-null-pointer main
- 4. Apply minimal fix; write a regression test to confirm the issue is resolved.
- 5. Open PR to main — fast-track review with 1 approver; CI must still pass.
- 6. Merge to main; tag: git tag -a v2.0.1
- 7. CI/CD deploys to production; verify monitoring dashboards return to normal.
- 8. Merge hotfix branch into develop to prevent the bug reappearing in the next release.
- 9. Write a post-mortem: root cause, impact, fix applied, prevention measures.

Q. How do you ensure hotfixes do not overwrite features?

- Always branch hotfix from main — never from develop (which may contain unfinished features).
- Merge the hotfix back into develop after applying to main to keep branches in sync.
- Small, focused hotfix PRs reduce the risk of unintended side effects.
- Run the full test suite even for hotfixes — urgency does not justify skipping CI.

Section 16 · Scenario-Based Questions

Q. Scenario 1: A critical bug is discovered in production.

- Branch to create: hotfix/critical-bug branched directly from main.
- Deploy fix: merge to main after CI passes; CI/CD pipeline deploys to production automatically.
- Merge back: apply hotfix to develop; cherry-pick to any active release branches if needed.
- Tag production: git tag -a v1.0.1 to mark the patched version.

Q. Scenario 2: Three developers modify the same file.

- Prevention: communicate ownership; use CODEOWNERS to assign reviewers for shared files.
- Use short-lived branches — merge to main daily to reduce divergence.
- Resolution: git pull --rebase origin main before opening a PR to detect conflicts early.
- When conflict arises: coordinate via PR comments; the last to merge resolves remaining conflicts.

Q. Scenario 3: A feature takes two months to complete.

- Risk of a single 2-month branch: massive merge conflicts, stale CI feedback.
- Better approach: break the feature into incremental sub-tasks, each with its own short-lived branch.
- Use feature flags to merge partial implementation into main safely (flag off = users do not see it).
- This keeps branches short-lived while allowing the feature to ship incrementally.

Q. Scenario 4: The main branch is broken.

- Immediate action: block all merges to main via branch protection until fixed.
- Identify the breaking commit: git bisect to binary-search the faulty commit.
- Revert: git revert — create a PR and merge quickly.
- Prevention: require all CI checks to pass before merge; add a smoke test to the PR pipeline.

Q. Scenario 5: A release fails in production.

- Rollback via Git: git revert on main; push; pipeline redeploys previous version.
- In GitOps: update image tag in manifest repo to last known good image; Argo CD syncs immediately.
- Kubernetes emergency: kubectl rollout undo deployment/app (use only if GitOps sync is too slow).
- Communicate: notify stakeholders via incident channel; post RCA within 24 hours.

Section 17 · Essential Git Commands

Q. git fetch vs git pull

- git fetch: downloads changes from remote but does NOT update local working branch — safe to run anytime.
- git pull: fetch + merge (or rebase) into current branch — updates your local branch.
- Best practice: git fetch && git log HEAD..origin/main to review changes before merging.

Q. What does git stash do?

- Temporarily saves uncommitted changes to a stack, leaving the working directory clean.
- Use case: urgent hotfix needed while mid-feature — stash work, fix the issue, restore with git stash pop.
- Commands: git stash, git stash pop, git stash list, git stash drop.

Q. What is git cherry-pick?

- Applies a specific commit from one branch onto another without merging the entire branch.
- Use case: a bug fix committed to develop needs to also go into main/release branch.
- Command: git cherry-pick
- Risk: can create duplicate commits and confusing history if overused; prefer full merges when possible.

Q. What is git revert?

- Creates a new commit that reverses the changes of a specified commit — history is preserved.
- Safe to use on shared or public branches because it does not rewrite history.
- Command: git revert
- Preferred for production rollbacks — auditable and collaborative.

Q. What is git reset?

- Moves the HEAD pointer and optionally modifies the staging area or working directory.
- git reset --soft HEAD~1: undo last commit, keep changes staged.
- git reset --mixed HEAD~1: undo last commit, keep changes unstaged (default behaviour).
- git reset --hard HEAD~1: undo last commit AND discard all changes — destructive, use carefully.
- Only use git reset on local commits not yet pushed to a remote branch.

Q. Difference between git reset and git revert

- git reset: rewrites history by moving HEAD backward — destructive on shared branches.
- git revert: creates a new commit to undo changes — safe for shared branches.
- Rule: use revert on public branches; use reset only on local uncommitted work.

Q. What is git reflog?

- Records all movements of HEAD — including commits, resets, checkouts, and rebases.
- Acts as a safety net: even after git reset --hard, reflog lets you recover lost commits.
- Command: git reflog — find the SHA of the lost commit, then git checkout .

Q. What is git tag? How do you create a release tag?

- Tags mark specific commits with a human-readable label — used for releases.
- Annotated tag (recommended): git tag -a v1.0.0 -m 'Production release 1.0.0'

- Push tags: git push origin v1.0.0 or git push --tags to push all tags.
- List tags: git tag -l; checkout a tag: git checkout v1.0.0 (creates detached HEAD).

Q. How do you restore a deleted branch?

- Find the last commit SHA: git reflog — look for the most recent commit on that branch.
- Recreate: git checkout -b recovered-branch
- On GitHub, deleted branches can be restored via the PR page ('Restore branch' button) within 30 days.

Section 18 · DevOps-Specific Branching

Q. How would you design a branching strategy for a microservices application?

- Each microservice has its own repository (polyrepo) or a clearly isolated module in a monorepo.
- Each service has an independent branching strategy and CI/CD pipeline.
- A shared manifest/config repo (watched by Argo CD) holds all service deployment configurations.
- Service A and Service B can be released independently — no shared release branches needed.

Q. Should infrastructure code follow the same branching strategy as application code?

- Yes, with adjustments: Terraform code in a dedicated infra-repo following Feature Branch Workflow.
- PRs against main trigger 'terraform plan' output as a CI step — reviewed before merge.
- Merge to main triggers 'terraform apply' — deploying infrastructure changes to the target environment.
- Environment separation: use Terraform workspaces or separate directories (envs/dev, envs/prod).

Q. How do you manage Kubernetes manifests in Git?

- Manifest repo structure: base/ (common configs) + overlays/ (dev, staging, prod via Kustomize).
- Or: charts/ (Helm) with per-environment values files: values-dev.yaml, values-prod.yaml.
- Argo CD Applications point to the appropriate overlay or values file for each environment.
- Image tags are the only thing changing between environments; updated by CI after Docker push.

Q. How would you handle production rollbacks in a GitOps environment?

- Revert the image-tag commit in the manifest repo: git revert HEAD && git push
- Argo CD detects the revert and re-deploys the previous image within seconds.
- Alternatively: use Argo CD's Rollback UI to select a previous successful sync revision.
- All rollbacks are recorded in Git history — fully auditable with who, what, and when.

Q. How do you manage secrets across environments?

- Never store secrets in Git — not even encrypted values (GitGuardian alerts catch this).
- Use AWS Secrets Manager or Parameter Store; inject into pods via External Secrets Operator.
- Kubernetes Secrets referenced in manifests are populated at runtime by the operator.
- Sealed Secrets (Bitnami) encrypt secrets for Git storage if secrets-in-Git is a hard requirement.

Q. How would you structure repos for Dev, QA, UAT, and Production?

- One manifest repo with Kustomize overlays: overlays/dev, overlays/qa, overlays/uat, overlays/prod.
- Four Argo CD Applications — each pointing to the same repo but a different overlay directory.
- Promotion: update image tag in overlays/qa, commit and push — Argo CD deploys to QA.
- Production promotion requires a PR with approval — enforced by branch protection on main.

Section 19 · Behavioural Questions (STAR Format)

Q. Tell me about a merge conflict you resolved.

- Situation: At Hisan Labs, two developers modified the same Terraform module for VPC configuration on separate feature branches.
- Task: Resolve the conflict without breaking the existing infrastructure remote state.
- Action: Identified conflicting lines in main.tf; coordinated with the second developer to understand their intent; manually merged both changes preserving VPC subnets and security group rules; ran terraform plan to validate no unintended resource changes.
- Result: Conflict resolved, PR merged, infrastructure applied cleanly with zero downtime.

Q. Describe a situation where a deployment failed because of a Git issue.

- Situation: A pipeline deployed a Docker image built from a stale feature branch instead of main due to an incorrect branch filter in the Jenkins webhook configuration.
- Action: Identified the misconfigured branch filter in the Jenkinsfile; corrected it to explicitly target main; added a build step that logs the Git commit SHA and branch name for audit.
- Result: Implemented as a mandatory log step across all 3 Jenkins pipelines — branch and SHA visible in every build log, preventing recurrence.

Q. Have you ever rejected a Pull Request? Why?

- Yes — a PR to the backend deployment pipeline included hardcoded AWS credentials in a Bash script.
- Action: Left a detailed review comment explaining the security risk (credential exposure in version history); suggested using IAM Roles for Service Accounts (IRSA) instead.
- Result: Developer updated the implementation to use IRSA; the fix aligned with the existing Kubernetes OIDC integration already in place on the EKS cluster.

Q. How do you ensure code quality before merging?

- Required CI status checks: SonarQube quality gate (coverage threshold, code smell limits), Trivy container scan (no CRITICAL vulnerabilities allowed).
- Minimum 1 reviewer approval; CODEOWNERS enforced for infrastructure changes.
- Conventional Commits format enforced to maintain a clean and consistent changelog.
- Stale approval dismissal — any new push to the branch invalidates previous approvals.

Q. Tell me about a mistake in source control and what you learned.

- Accidentally ran git push --force on a shared develop branch during a rebase, overwriting two team members' commits.
- Immediately used git reflog to recover the overwritten commits; cherry-picked them back onto develop.
- Lesson learned: always use git push --force-with-lease instead of --force — it checks that no one else has pushed since your last fetch.
- Added this rule to the team's Git guidelines document and raised it in the next sprint retrospective.

Section 20 · Advanced Questions (Senior DevOps Level)

Q. GitFlow vs Trunk-Based Development: Which would you choose and why?

- For a team with a scheduled release cycle (bi-weekly sprints, versioned APIs): GitFlow — it provides the structure needed for coordinated releases.
- For a team doing Continuous Deployment (multiple deploys per day): Trunk-Based Development — it removes the overhead of parallel long-lived branches.
- My recommendation: TBD with feature flags for greenfield microservices; GitFlow for stable, versioned products with external API consumers.
- Key factor: the team's deployment frequency and test automation maturity dictate the right choice.

Q. Why are many organisations moving away from GitFlow?

- GitFlow was designed for quarterly release cycles — most modern teams deploy daily or weekly.
- Long-lived branches create integration debt and slow down delivery.
- The rise of feature flags makes release branches unnecessary.
- CI/CD tooling works better with a single integration branch (trunk) than with 5+ branch types.
- The original GitFlow author himself acknowledged TBD may be more appropriate for CI environments.

Q. How does branching strategy affect deployment frequency?

- GitFlow: 1-2 deployments per sprint cycle; deployment gated by release branch lifecycle.
- Trunk-Based Development: multiple deployments per day are possible — every trunk commit can deploy.
- DORA metrics: elite DevOps performers deploy on-demand multiple times per day — all use trunk-based or equivalent approaches.
- Long-lived branches are the single biggest bottleneck to increasing deployment frequency.

Q. How do feature flags reduce branching complexity?

- Feature flags allow merging incomplete code to trunk safely — the code exists but is not activated.
- They eliminate the need for long-lived feature branches entirely.
- Enable A/B testing, canary releases, and instant rollbacks without any Git operations.
- Example: add a flag 'ENABLE_NEW_CHECKOUT_FLOW=false' in AWS AppConfig; set to true only when fully tested.

Q. How would you design a branching strategy for a Kubernetes GitOps platform?

- Two-repo model: app-repo (source code) and config-repo (Kubernetes manifests).
- App-repo: Feature Branch Workflow — short-lived feature branches, PRs to main, CI builds Docker image on merge.
- Config-repo: Kustomize overlays per environment (dev, staging, prod) all in main branch.
- CI in app-repo updates the image tag in config-repo via a commit after a successful Docker build.
- Argo CD Applications per environment watch their respective overlay; auto-sync for dev/staging, manual approval for prod.
- Branch protection on config-repo main: production changes require PR with 2 approvals.

Q. How would you handle infrastructure drift in a GitOps workflow?

- Argo CD continuously compares desired state (Git) with live state (Kubernetes cluster).
- Drift is shown as OutOfSync in the Argo CD dashboard and triggers alerts.
- With auto-sync enabled, Argo CD reverts drift automatically — enforcing immutability.

- For Terraform: use Atlantis or Terraform Cloud to detect and alert on plan drift; apply only via PR.
- Notifications: integrate Argo CD with Slack or PagerDuty so the team is aware of any drift immediately.

Q. How would you enforce Git standards across an organisation?

- Branch naming conventions enforced via GitHub branch rulesets (pattern matching on push).
- Conventional Commits enforced by commitlint in CI (pre-push hook or GitHub Actions check).
- PR templates standardise description, testing checklist, and impact assessment.
- CODEOWNERS ensures subject-matter experts review changes to critical infrastructure files.
- Periodic Git hygiene automation: stale branch cleanup scripts, required PR labels, protected tag policies.

Section * · Priority Topics — Focus Here First

#	Topic	Why It Matters for Your Profile
1	GitFlow vs TBD	Every senior DevOps interview asks this — have a clear position
2	Merge vs Rebase vs Squash	Tests depth of Git knowledge beyond basic commands
3	PR Process + Branch Protection	Directly linked to your Jenkins + GitHub Actions work
4	GitOps + Argo CD Branch Strategy	Core to your GitOps delivery pipeline project
5	GitHub Actions Branch Triggers	You have used this — answer with real project examples
6	Hotfix Workflow End-to-End	Scenario-based; walk through it step by step confidently
7	Feature Flags	Shows modern DevOps maturity beyond basic branching knowledge
8	Merge Conflict Resolution	Almost always asked — have a real story ready (STAR format)
9	CI/CD + Branch Integration	Ties your Jenkins + EKS + Argo CD experience together
10	Infrastructure Drift in GitOps	Advanced — differentiates you from other candidates